Evaluating LLM-Generated Code: Quality, Energy, and Cost Across Task Types

Simran Rao, Abdelrahman Azab, Rachana Devi, Daniel Ho-Yang Loh, and Nicolas Winarto
Department of Software Systems and Cybersecurity, Monash University
Melbourne, Australia

***Abstract*—With Large Language Models (LLMs) seeing massively increased usage in the creation and generation of code, it is easier now than ever before for programming to be done. However, the generated code invites questioning with regards to the quality, energy consumption and financial cost of generating code. While there currently exists research on benchmarks to measure these factors, it fails to reflect the environment of real-world software production. Studies that focus on energy consumption of code rely on proxy measures, such as floating point operations (FLOPs) or runtime, instead of directly measuring energy consumption in Joules. This is a major limitation, since many performance metrics like latency and memory usage do not necessarily correlate with energy efficiency, so a faster or smaller solution is not always a greener one. Even fewer account for the monetary cost of the inference process. In this project, LLM-generated code will be evaluated across three categories; algorithmic problems, data processing, and API integration. Six models are tested (three local: Qwen2.5-Coder 7B, DeepSeek-Coder-V2-Lite 16B, CodeLlama 13B; three API-based: GPT-4o mini, Claude Haiku 4.5, Gemini 2.0 Flash) alongside three prompt strategies (basic, efficiency-focused, and two-step). Generated solutions are compared against hand-coded optimal baselines across three measurement areas: code performance (execution time, CPU, memory, energy via CodeCarbon), code quality (unit test pass rate, redundancy), and cost and iteration overhead (token count, API latency, API cost). Initial results from Stage 1 indicate that for well-defined algorithmic and data processing tasks, the basic prompt consistently matches the accuracy of more elaborate strategies at significantly lower token cost. For API integration tasks, correctness degrades noticeably under two-step prompting. Full model comparison results across all six models are expected from Stage 2.**

***Index Terms*—large language models, code generation, energy efficiency, prompt engineering, code quality, software cost, benchmarking**

I. Introduction

THE growing adoption of large language models (LLMs) for automated code generation has introduced new questions about the quality, efficiency, and cost of the software they produce.

A. Functional Correctness

The existing literature on LLMs was initially heavily concerned with functional correctness benchmarks, such as HumanEval, a tool whose model generates short Python functions from docstrings which are then assessed via unit tests [4]. The approach was extended by MBPP, which synthesises basic Python programs from natural language [5], while APPS scaled evaluation to a larger set of problems with varying difficulty [6]. These benchmarks are narrow in scope as their tasks are short and algorithmic, and as such they are criticised for not accurately representing real-world software engineering scenarios. This led to the creation of more representative benchmarks such as NaturalCodeBench [7], and SWE-bench, which evaluates models against real GitHub issues within existing repositories [8].

B. Green AI Motivation

Another aspect of LLM usage is its energy consumption, which current literature argues ought to be as integral as correctness and accuracy when benchmarking LLMs. Similarly, Strubell et al. show that recent deep learning systems carry major financial and environmental costs, particularly in larger-scale experimentation settings [9]. These factors highlight how merely passing test cases is not indicative of a complete evaluation.

C. Energy Measurement

One technique for energy measurement is through Green Algorithms, which estimate the carbon footprint of computational processes including NLP algorithms and distributed computation across multiple CPUs [10]. Vartziotis et al. evaluate generated code using a broader set of sustainability metrics, including runtime, memory usage, floating point operations (FLOPs), GPU and CPU utilisation, energy consumption in Joules or MWh, power consumption in Watts, and CO_2 emissions [11]. Generated code can therefore be inefficient in two distinct ways: in the computational process used to generate it (inference cost), and in the execution of the code itself. The latter can be considerably more inefficient relative to human-written code, as Islam et al. demonstrated by evaluating 20 popular LLMs across 878 programming problems of varying difficulty and algorithmic category [1].

D. Prompt Engineering Techniques

Different prompts for the same task can affect all of the previously mentioned metrics, an area generally referred to as Prompt Engineering Techniques (PET). Structured Chain-of-Thought (SCoT) prompting has been shown to improve generated code performance by inducing the model to reason more carefully about program structure [12]. A prompt-refinement approach such as Prompt Alchemy similarly demonstrated that optimised prompts can improve benchmark performance

[13]. Such studies corroborate this paper's decision to account for PET in the analysis. Solovyeva and Castor introduce the concept of "babbling" in a 2026 study, using the term to describe output tokens that contribute nothing to the generated solution [14]. Their techniques for identifying and reducing such tokens may prove useful in PET analysis.

E. PET Impact on Energy

These studies on PET to date do not utilise energy cost as a metric, though PET may meaningfully affect the energy profile of generated code. A 2024 study by Cursaru et al. found that explicitly prompting LLMs to be energy efficient did not guarantee improved energy efficiency in the generated output [15]. A 2025 study by Rubei et al. showed that different PET can reduce energy consumption during LLM inference for a locally deployed Llama 3 model [16]. Notably, the former study concerned the energy efficiency of the generated code, while the latter concerned inference cost, highlighting that these are distinct and potentially conflicting dimensions.

F. Project Relevance

The studies reviewed above tend to address these topics independently of one another. PET and energy consumption are often studied in terms of inference cost and energy of the code generated itself but not simultaneously. This is important as the two are not independent: a prompting technique that lowers inference cost can lead to code that is significantly less energy efficient at runtime and vice versa. Existing evaluations also rely on narrow algorithmic benchmarks that are not the software tasks that programmers actually do every day, and the monetary cost of using API-based models is seldom considered. This project sits at the intersection of all three concerns, covering a range of tasks from algorithmic problems to data processing and API integration.

G. Aims, Objectives, and Research Questions

The main aim of this research is to combine the previously mentioned topics and evaluate the trade-off between code quality, inference energy, and execution energy with respect to different PET. As current research remains fragmented, for instance studies on PET and energy consumption tend to evaluate either inference cost or code execution energy but not both, the solutions to minimise one can affect the other. Such trade-offs are the main concern of this paper, which aims to unify evaluation of LLM-generated code in terms of correctness and energy cost, with consideration to PET. Our main objectives are as follows:

1) Compare local and API-based LLMs on a shared set of programming tasks spanning algorithmic, data processing, and API integration categories.
2) Evaluate generated code in terms of correctness, runtime, memory usage, and estimated execution energy.
3) Measure the resource cost of code generation via direct measurement for local models, and via token usage, API cost, latency, and iteration count for API-based models.
4) Evaluate three prompt strategies: basic, efficiency-focused, and two-step prompting.
5) Assess whether results are consistent across different tasks and models.

All of the above are centred around one major problem that this paper aims to address: current evaluation approaches to LLM-generated code treat correctness, energy, and cost as largely separate concerns, and the trade-offs between them remain poorly understood, particularly as a function of prompt strategy and task type. This motivates the following research questions:

- **RQ1:** How do different LLMs and prompt strategies affect the functional correctness and code quality of generated solutions across task types?
- **RQ2:** How do different LLMs and prompt strategies affect the execution energy and runtime of generated code relative to hand-coded baselines?
- **RQ3:** What is the relationship between inference cost and the quality and energy efficiency of the generated code?

II. Methodology

A. Overview

In this paper we evaluate LLM code for three different criteria, namely: functional correctness, token cost, and latency. Six programming tasks are used, spanning algorithmic problems, data processing, and API integration. Stage 1 holds the model fixed to isolate prompt strategy effects before Stage 2 expands to six models.

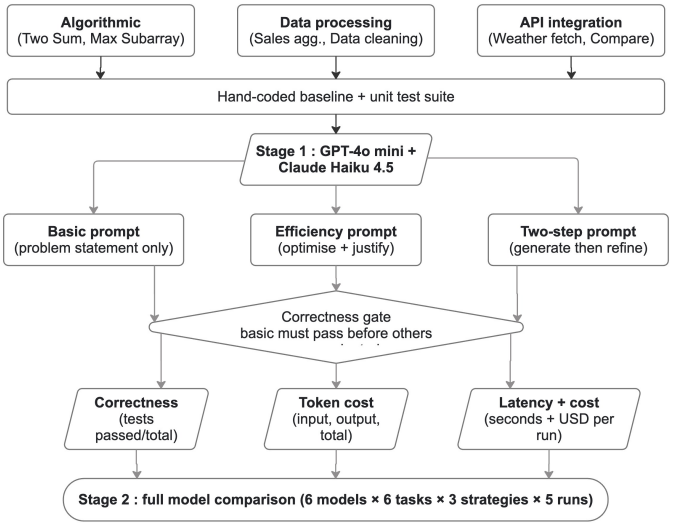


Fig. 1. Overview of the experimental methodology.

B. Task Design

To evaluate LLM-generated code across correctness, token cost, and latency, this study combines two API-based models with three prompt strategies across six programming tasks. The experiment is divided into two stages: ***Stage 1*** holds the model constant to isolate the effect of prompt strategy. ***Stage 2*** extends to a full six-model lineup once those findings are established.

TABLE I
MEASUREMENT METRICS BY MODEL TYPE

Metric	Description	How Measured	Applies To
Unit test pass rate	Tests passed / total	Hand-coded test suite	All models
Execution time	Time to run generated code	Python `time`	All models
Memory usage	Peak memory during execution	`psutil` RSS	All models
Energy consumption	Estimated energy during execution	CodeCarbon + `perf stat`	Local models
Token count	Input, output, total tokens	Provider API metadata	API models
Latency	Request to full response	Seconds around API call	API models
Estimated cost	Dollar cost per run	Published per-token pricing	API models

C. Models and Prompt Strategies

Six tasks were selected across three categories, each chosen to reflect a more realistic range of programming work than benchmarks such as HumanEval [4] or MBPP [5] typically cover. Two LeetCode algorithmic problems [6], Two Sum and Maximum Subarray, form the first category and serve as a controlled baseline given their well-known optimal solutions. The second category consists of two data processing tasks inspired by Kaggle scenarios, sales aggregation and data cleaning, which introduce the kind of practical, messier inputs that short algorithmic problems tend to abstract away. The third category covers two API integration tasks built around the OpenWeatherMap API, a single-city weather fetch and a multi-city comparison, adding error handling and structured JSON parsing that are absent from most standard evaluations. Each task was paired with a hand-coded baseline solution and a unit test suite prepared before any model was queried, ensuring the correctness standard was set independently of what any model produced. To illustrate the difference between strategies, the following shows the prompts applied to the Two Sum problem. The basic prompt passes only the problem statement with a minimal system instruction:

```
System: You are an expert software engineer.
        Provide a Python solution.
User:   Solve this: [problem statement]
```

The efficiency-focused and two-step prompts follow the same structure but extend the user message with explicit optimisation instructions and an iterative refinement step respectively.

D. Measurement

In Stage 1 we evaluate GPT-4o mini from OpenAI [17] and Claude Haiku 4.5 from Anthropic [18]. These are lightweight cost-efficient tiers of their respective families, reflecting the kind of models a developer would realistically reach for in day-to-day use. Temperature was fixed at 0.0 across all runs to keep outputs as deterministic as possible and ensure that differences between conditions reflect the prompt rather than sampling variation. Three strategies were applied to every task:

- The basic prompt provides only the problem statement and a minimal instruction to produce a Python solution, acting as the correctness gate since the remaining two strategies are only evaluated where this one first passes.
- The efficiency-focused prompt retains the same problem statement but additionally asks the model to optimise for runtime and memory and justify its algorithmic choice, directly testing whether that kind of explicit instruction changes what the model produces.
- The two-step prompt splits generation across two API calls, first producing an initial solution then feeding it back asking for one focused pass of targeted improvements without a full rewrite, mirroring an iterative workflow a developer might use in practice.

E. Experimental Stages

Each condition was run five times with a short delay between calls to reduce rate limiting interference. Per run, input tokens, output tokens, total tokens, end-to-end latency, and estimated dollar cost were recorded using each provider's published pricing, with two-step runs summing both calls to give an honest total overhead. Correctness was assessed immediately after each run by executing the extracted code against the unit test suite, recording tests passed out of total per run rather than aggregating upfront. This preserves run-level variance in the results, which proved particularly relevant for API integration tasks where the same condition produced noticeably different outputs across consecutive runs despite the fixed temperature. All raw per-run results for Stage 1 are available online.[1]

III. INITIAL RESULTS

Stage 1 results are available across all three task categories using GPT-4o mini and Claude Haiku 4.5. Key findings are summarised below.

A. Algorithmic Tasks

All 30 runs across both LeetCode tasks achieved 100% unit test pass rate regardless of prompt strategy.

The basic prompt was the fastest and most token-efficient, averaging 16.29s latency and 269 tokens on Two Sum. The efficiency-focused and two-step prompts used 1.9-2.6 times more tokens with no improvement in correctness, suggesting the model already produces near-optimal solutions for well-known problems without additional prompting. The following snippet shows a representative solution generated by GPT-4o mini for the Two Sum problem under the basic prompt:

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

The model consistently produced a hash map approach with O(n) time and O(n) space complexity, matching the known optimal solution. The solution is concise, contains no redundant code, and completes the problem in a single pass without nested iteration.

[1]Raw Stage 1 results: Click here to view

TABLE II
STAGE 1 AVERAGED RESULTS (5 RUNS × 2 TASKS PER CATEGORY): CORRECTNESS, TOKEN USAGE, AND COST BY PROMPT STRATEGY AND MODEL

Task Category	Prompt Strategy	Pass Rate	Tokens (GPT)	Tokens (Claude)	Avg Cost (GPT/Claude)
Algorithmic Two Sum + Max Subarray	Basic	30/30	314	795	$0.00106/$0.00358
	Efficiency	30/30	444	998	$0.00151/$0.00445
	Two-step	30/30	763	1,966	$0.00211/$0.00638
Data Processing Sales Agg + Data Cleaning	Basic	30/30	805	1,321	$0.00231/$0.00508
	Efficiency	30/30	996	1,524	$0.00304/$0.00594
	Two-step	30/30	1,645	3,157	$0.00420/$0.01031
API Integration Weather Fetch + Compare	Basic	25/30	557	1,360	$0.00170/$0.00585
	Efficiency	23/30	714	1,469	$0.00225/$0.00621
	Two-step	18/30	1,596	4,169	$0.00488/$0.01503

B. API Integration Tasks

Results varied more significantly across prompt strategies for API tasks. For the easier single-city weather fetch task, OpenAI's two-step strategy was the only configuration to achieve a perfect pass rate (15/15). Anthropic's model scored 10/15 across all three strategies. For the harder multi-city comparison task, OpenAI's basic prompt achieved 15/15, while the two-step strategy degraded to 3/15. Anthropic's model scored only 5/15 on basic and two-step, suggesting the two-step approach introduces inconsistencies in multi-step API logic. Token costs for Anthropic were consistently 2–3 times higher than OpenAI across all strategies, driven by longer output responses rather than input length.

C. Anticipated Stage 2 Results

Based on Stage 1 findings, we anticipate that the basic prompt will remain competitive or superior for well-defined tasks across all six models. Local models are expected to show higher absolute energy costs per run due to inference-side measurement, while API models will be evaluated on a cost-per-correct-solution basis. Task type is expected to remain a significant moderating factor, with API integration tasks showing the greatest variation across models and strategies.

IV. FUTURE WORK PLANS

Future plans for the project include:

- **Expanded model lineup with direct energy measurement.** Three local models (Qwen2.5-Coder 7B, DeepSeek-Coder-V2-Lite 16B, CodeLlama 13B) and Gemini 2.0 Flash will be added, with CodeCarbon recording direct energy consumption in Joules for local inference.
- **More complex task types.** The task set will be extended to include multi-file generation and database interaction scenarios, which introduce dependencies and state management that single-function tasks do not capture and are more representative of real-world development work.
- **Energy payback analysis.** The study will investigate the energy payback point: how many executions of a piece of LLM-generated code are required before the energy saved through efficient generated code offsets the inference cost of generating it. This framing connects inference overhead directly to the long-term sustainability of using LLMs in software development.

GENAI USAGE

GenAI tools including Claude and ChatGPT were used to assist with formatting and rewording portions of this paper.

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology at Monash University for supporting this work, and Professor John Grundy for his supervision and guidance throughout the project.

REFERENCES

[1] M. A. Islam, D. V. Jonnala, R. Rekhi, P. Pokharel, S. Cilamkoti, A. Imran, T. Kosar, and B. Turkkan, "Evaluating the energy-efficiency of the code generated by LLMs," *arXiv preprint arXiv:2505.20324*, May 2025.

[2] H. Peng, A. Gupte, N. J. Eliopoulos, C. C. Ho, R. Mantri, L. Deng, W. Jiang, Y.-H. Lu, K. Läufer, G. K. Thiruvathukal, and J. C. Davis, "Large language models for energy-efficient code: Emerging results and future directions," *arXiv preprint arXiv:2410.09241*, Oct. 2024.

[3] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *arXiv preprint arXiv:1907.10597*, Jul. 2019.

[4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, Jul. 2021.

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, Aug. 2021.

[6] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS) Track Datasets Benchmarks*, 2021.

[7] S. Zhang, H. Zhao, X. Liu, Q. Zheng, Z. Qi, X. Gu, Y. Dong, and J. Tang, "NaturalCodeBench: Examining coding performance mismatch on HumanEval and natural user queries," in *Findings of the Assoc. Comput. Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 7907–7928.

[8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[9] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Florence, Italy, Jul. 2019, pp. 3645–3650.

[10] L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," *Advanced Science*, vol. 8, no. 12, p. 2100707, Jun. 2021.

[11] T. Vartziotis, I. Dellatolas, G. Dasoulas, M. Schmidt, F. Schneider, T. Hoffmann, S. Kotsopoulos, and M. Keckeisen, "Learn to code sustainably: An empirical study on LLM-based green code generation," in *Proc. 1st Int. Workshop Large Language Models for Code (LLM4Code '24)*, ACM, 2024, pp. 30–37.

[12] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, pp. 1–23, 2023.

[13] S. Ye, Z. Sun, G. Wang, L. Guo, Q. Liang, Z. Li, and Y. Liu, "Prompt alchemy: Automatic prompt refinement for enhancing code generation," *arXiv preprint arXiv:2503.11085*, Mar. 2025.

[14] L. Solovyeva and F. Castor, "Babbling suppression: Making LLMs greener one token at a time," *arXiv preprint arXiv:2604.06755*, 2026.

[15] V.-A. Cursaru, L. Duits, J. Milligan, D. Ural, B. R. Sanchez, V. Stoico, and I. Malavolta, "A controlled experiment on the energy efficiency of the source code generated by Code Llama," in *Proc. 17th Int. Conf. Quality Inf. Commun. Technol. (QUATIC 2024)*, Commun. Comput. Inf. Sci., vol. 2178. Springer, 2024, pp. 161–176.

[16] R. Rubei, A. Moussaid, C. Di Sipio, and D. Di Ruscio, "Prompt engineering and its implications on the energy consumption of large language models," *arXiv preprint arXiv:2501.05899*, Jan. 2025.

[17] OpenAI, "GPT-4o mini system card," OpenAI, 2024. [Online]. Available: https://openai.com/index/gpt-4o-mini

[18] Anthropic, "Claude Haiku model card," Anthropic, 2025. [Online]. Available: https://www.anthropic.com/claude/haiku